PROJECT REPORT

USERNAME AND EMAIL VALIDATION SYSTEM

Using Regular Expressions (Regex), Python Functions and GUI

Submitted to: Department of Information Technology

A D Patel Institute of Technology

The Charutar Vidya Mandal University

Academic Year: 2026–27

Project developed using Python, re module and Tkinter

ABSTRACT

The Username and Email Validation System is a Python desktop application that checks whether a student's enrollment number and college email address follow the required format. The project uses the built-in re module for regular-expression matching and Tkinter to provide a graphical user interface. The enrollment number must contain exactly 10 digits, while the email address must use the @adit.ac.in college domain. The program displays clear error messages for empty or invalid inputs and a success alert when both fields are valid.

1. INTRODUCTION

Input validation is an important part of software development. It helps a program check data before accepting or processing it. Incorrect or incomplete information can cause errors and make a system harder to use. This project demonstrates a simple student-login form in which two fields are checked before the application displays a success message.

2. PROBLEM STATEMENT

A student portal needs to verify that the entered enrollment number and college email address match the expected formats. Without validation, users may enter an enrollment number with the wrong number of digits or use a personal email address instead of the college email address. The project addresses this problem through regular expressions and user-friendly GUI feedback.

3. OBJECTIVES

- Create a simple desktop login form using Tkinter.
- Validate an enrollment number containing exactly 10 digits.
- Validate that the email address ends with @adit.ac.in.
- Show useful messages when fields are empty or invalid.
- Display a success alert when both inputs pass validation.
- Provide a Clear button to reset the fields and error labels.

4. TOOLS AND TECHNOLOGIES

- Python: main programming language used to implement the application.
- re module: supports regular-expression matching.
- Tkinter: Python GUI library used for labels, text-entry fields, buttons, frames and alert windows.
- Python functions: separate validation, alert, submit-handling and clear-field tasks into reusable units.

5. SYSTEM DESIGN AND WORKFLOW

The application follows this sequence:

- The user opens the Student Login window.
- The user enters an enrollment number and college email address.
- When Submit is clicked, the program reads both entries and clears previous error labels.
- The enrollment number is checked against the 10-digit pattern.
- The email is checked against the required ADIT domain pattern.
- If one or more checks fail, the program displays the relevant errors.
- If both checks pass, a success alert displays the entered details.
- The Clear button removes the values and error messages.

6. REGULAR-EXPRESSION RULES

```
Enrollment pattern:
r"^\d{10}$"

Email pattern:
r"^[a-zA-Z0-9._%+-]+@adit\.ac\.in$"
```

In the enrollment pattern, ^ marks the beginning, \d represents a digit, {10} requires exactly ten digits, and $ marks the end. In the email pattern, the character class before @ allows letters, digits and selected email characters. The escaped dot in adit\.ac\.in matches a literal period. The anchors require the whole input to follow the pattern.

7. IMPORTANT CODE SNIPPETS

7.1 Enrollment validation

```
def validate_enrollment(enrollment: str) -> bool:
    pattern = r"^\d{10}$"
    return bool(re.fullmatch(pattern, enrollment.strip()))
```

The function removes leading and trailing spaces and uses re.fullmatch() to check the complete input. It returns True only when the input contains exactly ten digits.

7.2 College email validation

```
def validate_email(email: str) -> bool:
    pattern = r"^[a-zA-Z0-9._%+-]+@adit\.ac\.in$"
    return bool(re.fullmatch(
        pattern, email.strip(), re.IGNORECASE
    ))
```

The email function checks the permitted characters and the required college domain. The re.IGNORECASE flag makes matching case-insensitive.

7.3 Submit and error handling

```
enrollment_input = entry_enrollment.get().strip()
email_input = entry_email.get().strip()
errors = []

if not enrollment_input:
    errors.append("- Enter Enrollment number.")
elif not validate_enrollment(enrollment_input):
    errors.append("- Invalid Enrollment Number: Must be 10 digits.")

if not email_input:
    errors.append("- Enter College Email.")
elif not validate_email(email_input):
    errors.append("- Invalid College Email: Must end with '@adit.ac.in'.")

if errors:
    show_alert("Validation Failed", "\n\n".join(errors))
else:
    show_alert("Success",
        f"Welcome !\n\nUser: {enrollment_input}\nEmail: {email_input}",
        is_success=True)
```

The handler checks both fields and collects validation errors. The program does not show the success alert unless the error list is empty. The full source also defines show_alert() for formatted popup messages and clear_fields() for resetting the form.

8. GRAPHICAL USER INTERFACE AND OUTPUT

The main window is built with Tkinter. It contains a Student Login heading, an enrollment-number entry field, a college-email entry field, error labels, and Submit and Clear buttons.

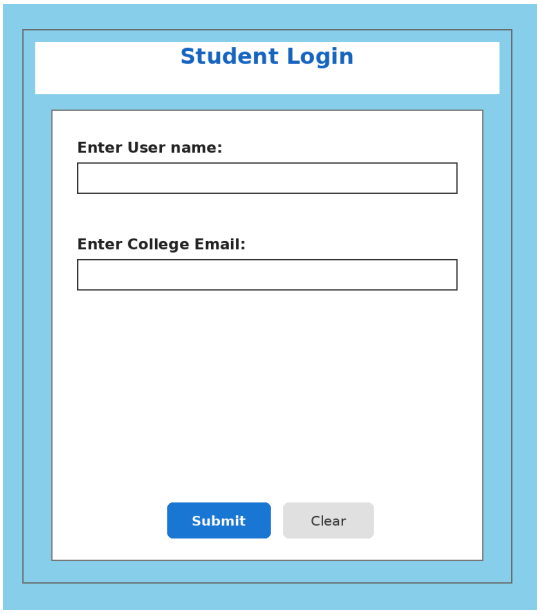


Figure 1. Representative view of the initial Student Login form.

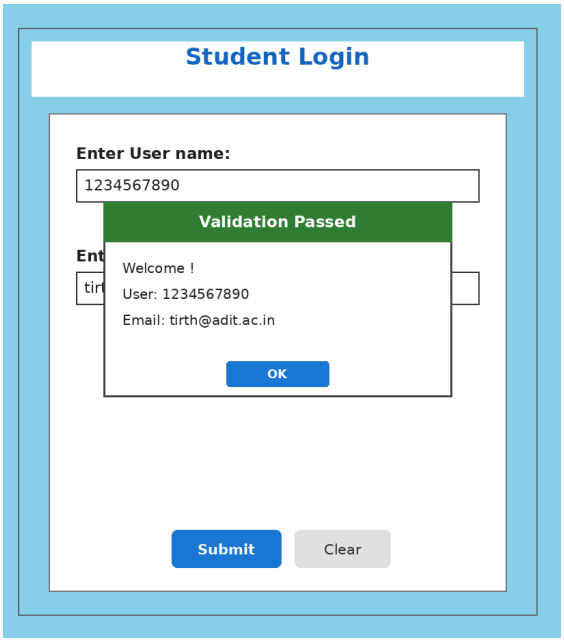


Figure 2. Representative successful-validation output.

8.1 Successful validation

For example, enrollment number 1234567890 and email tirth@adit.ac.in satisfy the patterns used by the program. The success popup displays a welcome message and the submitted values.

9. TESTING AND RESULTS

Test case	Enrollment	Email	Expected result
Valid inputs	1234567890	tirth@adit.ac.in	Validation passed
Wrong digit count	12345	tirth@adit.ac.in	Enrollment error
Wrong email domain	1234567890	student@gmail.com	Email error
Empty enrollment	(empty)	tirth@adit.ac.in	Required-field error
Empty email	1234567890	(empty)	Required-field error

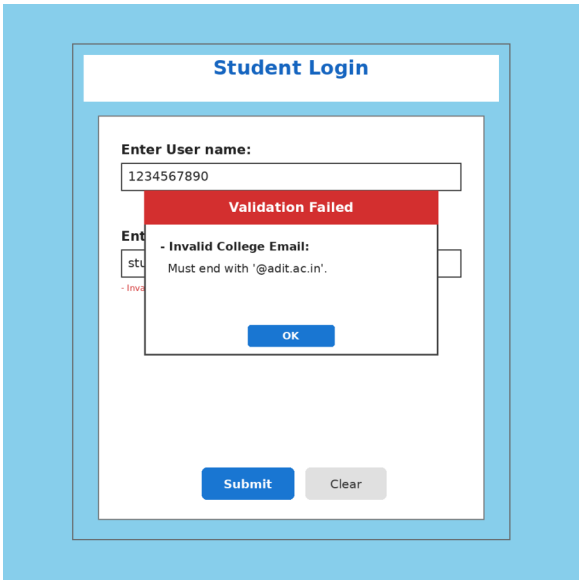


Figure 3. Representative invalid-email output.

10. ADVANTAGES AND LIMITATIONS

Advantages

- Simple graphical interface suitable for a beginner-level project.
- Validation logic is separated into reusable functions.
- Immediate feedback helps users correct invalid input.
- Both fields are checked before success is displayed.

Limitations

- The project checks format only; it does not verify whether an enrollment number actually belongs to a student.
- It does not check whether the email inbox exists or whether the user owns the address.
- The current enrollment rule requires exactly 10 digits and the email rule restricts the domain to @adit.ac.in.

11. CONCLUSION

The Username and Email Validation System demonstrates the use of Python regular expressions, functions and Tkinter GUI components in a practical application. It checks a 10-digit enrollment number and an ADIT college email address, displays clear validation errors, and provides a success message for valid input. Future improvements could include connecting the form to a database, adding secure authentication, and verifying email ownership.

REFERENCE

Project source code: python project.py

Python documentation: re module and tkinter module.